

TUGAS PRAKTIKUM PERTEMUAN 11

Nama: Dede Risma Komalasari

NIM: J0403251049

Kelas: TPL A2

1. Praktikum 1: Membuat Adjacency Matrix
Implementasi Kode

```
def create_graph_matrix(V, edges):
    # Membuat matriks 2D diisi dengan 0 (matriks V x V)
    mat = [[0 for _ in range(V)] for _ in range(V)]

    # Mengisi matriks berdasarkan list edges
    for u, v in edges:
        mat[u][v] = 1
        mat[v][u] = 1 # Karena graph tidak berarah (undirected)
    return mat

if __name__ == "__main__":
    V = 4
    # 0 terhubung ke 1 dan 2; 2 terhubung ke 1 dan 3
    edges = [[0, 1], [0, 2], [1, 2], [2, 3]]

    matrix = create_graph_matrix(V, edges)

    # Menampilkan Adjacency Matrix
    print("Adjacency Matrix Representation:")
    for row in matrix:
        print(row)
```

Output:

```
PS D:\dokumen\Algoritma-struktur-data-pertemuan1> & C:\Users\ThinkPad\AppData\
e "d:/dokumen/Algoritma-struktur-data-pertemuan1/pertemuan 11/praktikum1.py"
Adjacency Matrix Representation:
[0, 1, 1, 0]
[1, 0, 1, 0]
[1, 1, 0, 1]
[0, 0, 1, 0]
PS D:\dokumen\Algoritma-struktur-data-pertemuan1> █
```

2. Prakrium 2: Membuat Adjacency List

Implementasi Kode:

```
# =====  
# Dede Risma Komalasari  
# J0403251049  
# =====  
  
# Representasi Graph menggunakan Dictionary Python  
✓ graph = {  
    "A": ["B", "C"],  
    "B": ["A", "D"],  
    "C": ["A", "D"],  
    "D": ["B", "C"]  
}  
  
✓ def display_adjacency_list(adj_list):  
    print("Adjacency List Representation:")  
    ✓ for node, neighbors in adj_list.items():  
        print(f"{node}: {' '.join(neighbors)}")  
  
✓ if __name__ == "__main__":  
    display_adjacency_list(graph)
```

Output:

```
e "d:/dokumen/Algoritma-struktur-data-pertemuan1/pertemuan 11/praktikum2.py"  
Adjacency List Representation:  
A: B C  
B: A D  
C: A D  
D: B C  
PS D:\dokumen\Algoritma-struktur-data-pertemuan1>
```

3. Praktikum 3: Konversi Adjacency Matrix ke Adjacency List

Implementasi Kode:

```

matrix = [
    [0, 1, 1, 0],
    [1, 0, 1, 0],
    [1, 1, 0, 1],
    [0, 0, 1, 0]
]

def convert_matrix_to_list(mat):
    adj_list = {}
    v = len(mat)

    for i in range(v):
        neighbors = []
        for j in range(v):
            if mat[i][j] == 1:
                neighbors.append(j)
        adj_list[i] = neighbors
    return adj_list

if __name__ == "__main__":
    result_list = convert_matrix_to_list(matrix)

    print("Hasil Konversi ke Adjacency List:")
    for node, neighbors in result_list.items():
        print(f"{node}: {neighbors}")

```

Output:

```

PS D:\dokumen\Algoritma-struktur-data-pertemuan11> python D:\dokumen\Algoritma-struktur-data-pertemuan11\pertemuan11\praktikum3.py
Hasil Konversi ke Adjacency List:
0: [1, 2]
1: [0, 2]
2: [0, 1, 3]
3: [2]
PS D:\dokumen\Algoritma-struktur-data-pertemuan11>

```

Konversi dilakukan untuk mengubah representasi matrix menjadi list. Proses ini dilakukan dengan membaca setiap baris pada matrix dan mencatat node yang memiliki nilai 1 sebagai tetangga.

4. Praktikum 4: Studi Kasus Dunia Nyata - Peta Kota
Nim akhir: 9

a. Source code

```
def tampilkan_matrix(matrix, nodes=None):
    """Menampilkan adjacency matrix dengan label node."""
    if nodes is None:
        nodes = list(range(len(matrix)))

    print("    " + " ".join(str(node) for node in nodes))
    for node, row in zip(nodes, matrix):
        print(f"{node} : " + " ".join(str(value) for value in row))

def tampilkan_adjacency_list(graph):
    """Menampilkan adjacency list."""
    for node, neighbors in graph.items():
        print(f"{node} -> {neighbors}")

def buat_matrix_tidak_berarah(nodes, edges):
    """Membuat adjacency matrix untuk graph tidak berarah."""
    index = {node: i for i, node in enumerate(nodes)}
    matrix = [[0 for _ in nodes] for _ in nodes]

    for asal, tujuan in edges:
        i = index[asal]
        j = index[tujuan]
        matrix[i][j] = 1
        matrix[j][i] = 1

    return matrix

def buat_list_tidak_berarah(nodes, edges):
    """Membuat adjacency list untuk graph tidak berarah."""
    graph = {node: [] for node in nodes}

    for asal, tujuan in edges:
        graph[asal].append(tujuan)
        graph[tujuan].append(asal)

    return graph

print("Digit akhir NIM: 9")
print("Studi kasus : Peta Kota")

print("Digit akhir NIM: 9")
print("Studi kasus : Peta Kota")

nodes_kota = ["Bogor", "Depok", "Jakarta", "Bandung", "Sukabumi"]
edges_kota = [
    ("Bogor", "Depok"),
    ("Bogor", "Sukabumi"),
    ("Depok", "Jakarta"),
    ("Depok", "Sukabumi"),
    ("Jakarta", "Bandung"),
    ("Bandung", "Sukabumi"),
]

graph_kota = buat_list_tidak_berarah(nodes_kota, edges_kota)
matrix_kota = buat_matrix_tidak_berarah(nodes_kota, edges_kota)

print("\nNode / Vertex:")
for kota in nodes_kota:
    print(f"- {kota}")

print("\nEdge / Hubungan antar kota:")
for asal, tujuan in edges_kota:
    print(f"- {asal} terhubung dengan {tujuan}")

print("\nAdjacency List Peta Kota:")
tampilkan_adjacency_list(graph_kota)

print("\nAdjacency Matrix Peta Kota:")
tampilkan_matrix(matrix_kota, nodes_kota)
```

b. Hasil Program

```

PS D:\dokumen\Algoritma-struktur-data-pertemuan1> & C:\Users\ThinkPad\
51049_Dede Risma Komalasari.py"
Digit akhir NIM: 9
Studi kasus : Peta Kota

Node / Vertex:
- Bogor
- Depok
- Jakarta
- Bandung
- Sukabumi

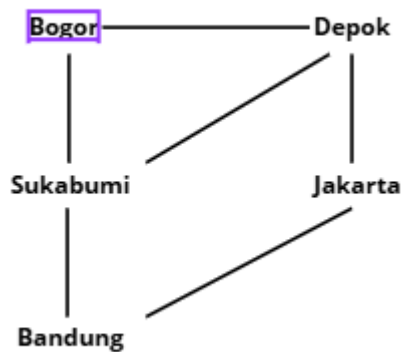
Edge / Hubungan antar kota:
- Bogor terhubung dengan Depok
- Bogor terhubung dengan Sukabumi
- Depok terhubung dengan Jakarta
- Depok terhubung dengan Sukabumi
- Jakarta terhubung dengan Bandung
- Bandung terhubung dengan Sukabumi

Adjacency List Peta Kota:
Bogor -> ['Depok', 'Sukabumi']
Depok -> ['Bogor', 'Jakarta', 'Sukabumi']
Jakarta -> ['Depok', 'Bandung']
Bandung -> ['Jakarta', 'Sukabumi']
Sukabumi -> ['Bogor', 'Depok', 'Bandung']

Adjacency Matrix Peta Kota:
    Bogor Depok Jakarta Bandung Sukabumi
Bogor : 0 1 0 0 1
Depok : 1 0 1 0 1
Jakarta : 0 1 0 1 0
Bandung : 0 0 1 0 1
Sukabumi : 1 1 0 1 0
PS D:\dokumen\Algoritma-struktur-data-pertemuan1>

```

c. Desain Graph



d. Penjelasan Node dan Edge

1) Node

Bogor, Depok, Jakarta, Bandung, dan Sukabumi

2) Edge (hubungan):

Bogor terhubung dengan Depok

Bogor terhubung dengan Sukabumi

Depok terhubung dengan Jakarta

Depok terhubung dengan Sukabumi

Jakarta terhubung dengan Bandung

Bandung terhubung dengan Sukabumi

Graph ini bersifat tidak berarah, sehingga setiap hubungan berlaku dua arah.

e. Analisis adjacency list vs matrix

1) Adjacency List

Adjacency Matrix merepresentasikan peta kota dalam bentuk tabel dua dimensi, di mana setiap baris dan kolom mewakili kota-kota yang ada. Hubungan antar kota ditandai dengan angka 1 jika terdapat jalur langsung dan angka 0 jika tidak ada.

Struktur ini sangat unggul dalam hal kecepatan akses, karena kita dapat mengetahui apakah dua kota terhubung secara instan hanya dengan melihat titik potong antara baris dan kolomnya. Namun, metode ini memiliki kelemahan pada efisiensi memori, karena program harus menyediakan ruang untuk seluruh kombinasi kota yang mungkin, meskipun pada kenyataannya banyak kota yang tidak memiliki jalur langsung. Metode ini lebih cocok digunakan jika hampir semua kota dalam peta saling terhubung satu sama lain.

2) Adjacency Matrix

Adjacency List merepresentasikan peta kota dalam bentuk daftar koleksi, di mana setiap kota hanya mencatat kota-kota lain yang menjadi tetangga langsungnya. Berbeda dengan matriks, daftar ini tidak menyimpan informasi tentang kota yang tidak terhubung.

Kelebihan utama dari metode ini adalah kehematan penggunaan memori, karena ruang penyimpanan hanya digunakan untuk mencatat jalur yang benar-benar ada. Selain itu, daftar ini sangat efektif saat kita ingin melakukan penelusuran rute, karena program bisa langsung mengetahui daftar tetangga suatu kota tanpa harus memindai seluruh data kota lainnya. Kelemahannya hanya terletak pada sedikit keterlambatan saat kita ingin memastikan hubungan antara dua kota tertentu, karena program harus mencari di dalam daftar kota tersebut terlebih dahulu.

f. Kesimpulan singkat

Representasi graph dapat dilakukan menggunakan adjacency matrix maupun adjacency list. Pada studi kasus peta kota, adjacency list lebih efisien digunakan karena jumlah hubungan antar kota relatif sedikit sehingga lebih hemat memori dan fleksibel. Sementara itu, adjacency matrix lebih unggul dalam kecepatan akses, namun kurang optimal untuk graph berukuran besar dengan koneksi yang tidak terlalu banyak.